OSSP-WEEK2-PRACTICAL

Task1: Develop a C program that uses the system calls open(), read(), write(), and close() to copy the contents of one file to another. Explain how control transitions between user space and kernel space during execution.

Source code:

```c
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdlib.h>

int main() {
    int source, destination;
    char buffer[1024];
    ssize_t bytesRead, bytesWritten;

    source = open("source.txt", O_RDONLY);

    if (source < 0) {
        perror("Error opening source file");
        return 1;
    }

    destination = open("destination.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (destination < 0) {
        perror("Error opening destination file");
        close(source);
        return 1;
    }

    printf("Source file opened successfully.\n");
    printf("Destination file opened successfully.\n");

    while ((bytesRead = read(source, buffer, sizeof(buffer))) > 0) {
        bytesWritten = write(destination, buffer, bytesRead);

        if (bytesWritten != bytesRead) {
            perror("Error writing to destination file");
            close(source);
            close(destination);
            return 1;
        }
    }

    if (bytesRead < 0) {
        perror("Error reading source file");
    } else {
        printf("File contents copied successfully.\n");
    }

    close(source);
    close(destination);

    printf("Files closed successfully.\n");

    return 0;
}
```

Output:

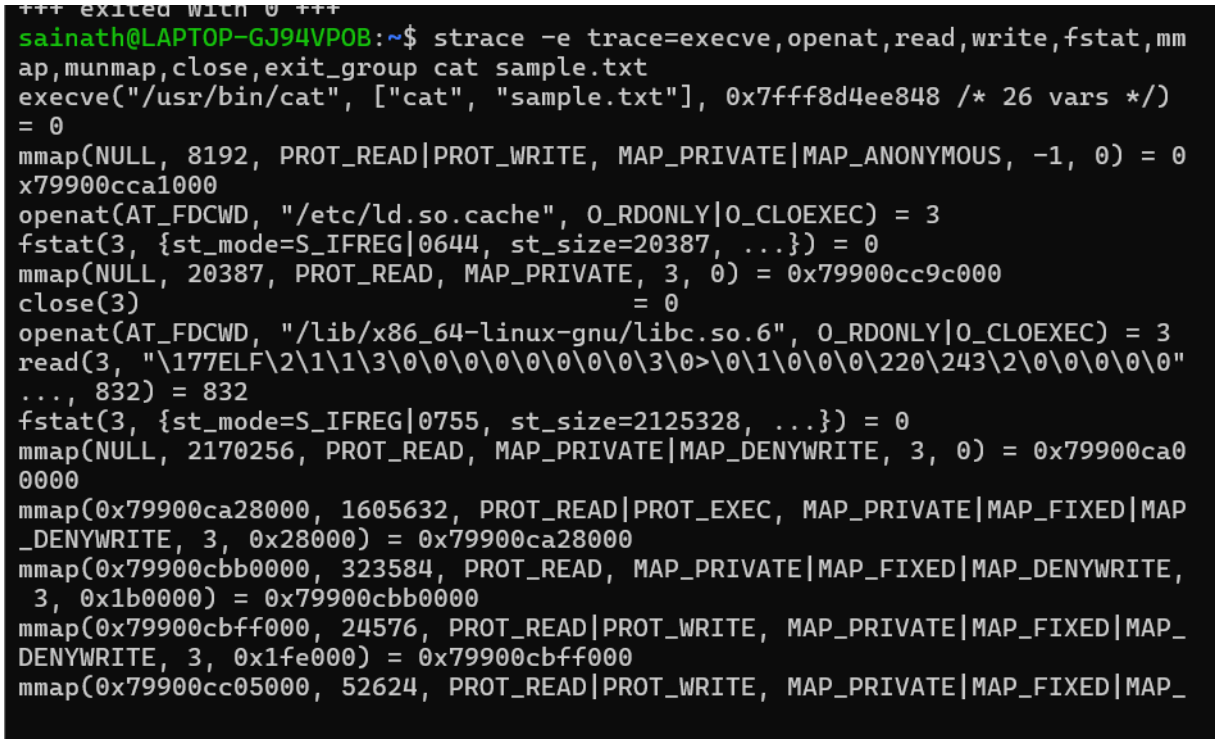
```
 fork-example.c.save          task3.c
sainath@LAPTOP-GJ94VPOB:~$ ./week2_task1
Source file opened successfully.
Destination file opened successfully.
File contents copied successfully.
Files closed successfully.
sainath@LAPTOP-GJ94VPOB:~$
```

Observation: I learned how to use the Linux system calls open(), read(), write(), and close() to copy the contents of one file to another. I understood how data is read from the source file into a buffer and then written to the destination file. I also learned how control moves from user space to kernel space when a system call is made, and how the operating system manages file resources during the file-copy operation.

Task2: Use the strace utility to trace all system calls generated by the command cat sample.txt. Analyze the sequence of system calls and identify the kernel services involved.

Command: strace cat sample.txt

Output:



Observation: I learned how to use the strace utility to trace the system calls generated by the cat command. I learned how execve() starts the program, openat() opens the file, read() reads the file contents, and write() displays them on the terminal. I also learned how fstat(), mmap(), munmap(), close(), and exit_group() support file access, memory management, resource cleanup, and process termination.